

```
---
title: "Applied Data Visualizations Portfolio"
author: "Kumar Duvvapu"
date: today
format:
  html:
    toc: true
    theme: cosmo
---

## Busiest Airports Analysis

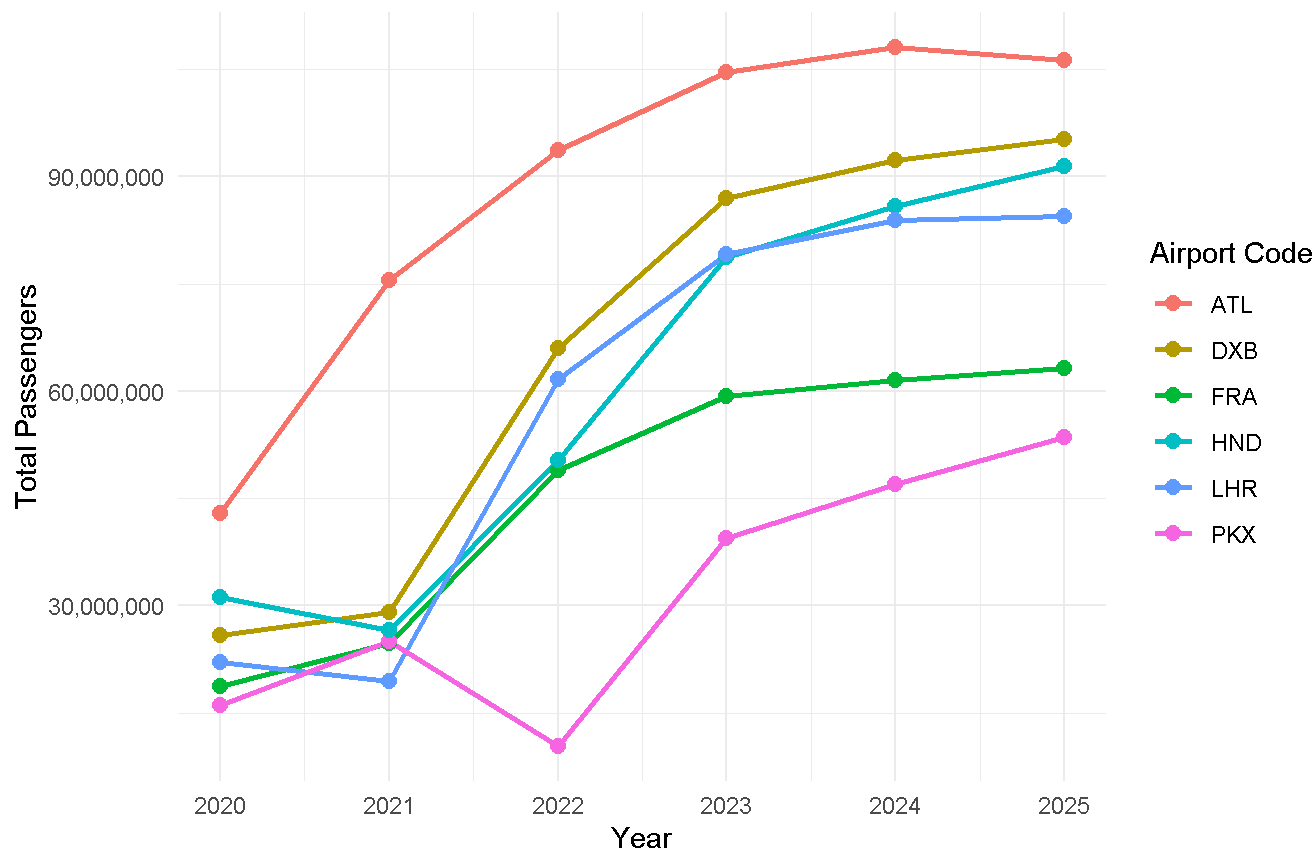
The aviation industry experienced a severe global disruption followed by a tiered recovery, which

Looking at the data, Atlanta (ATL) stands out as the dominant outlier in sheer volume. As a massiv
```

Passenger Traffic at Six Major Global Airports (2020-2025)

airport	2020	2021	2022	2023	2024	2025
ATL	42,918,685	75,537,213	93,699,630	104,653,451	108,067,766	106,302,208
DXB	25,831,363	29,110,609	66,069,981	86,994,365	92,331,506	95,200,000
HND	31,135,000	26,563,000	50,330,000	78,719,586	85,900,167	91,430,000
LHR	22,109,726	19,393,145	61,614,508	79,183,366	83,884,572	84,463,000
FRA	18,770,998	24,800,998	48,918,118	59,355,389	61,564,957	63,189,666
PKX	16,091,449	25,051,012	10,277,323	39,410,521	47,000,000	53,618,949

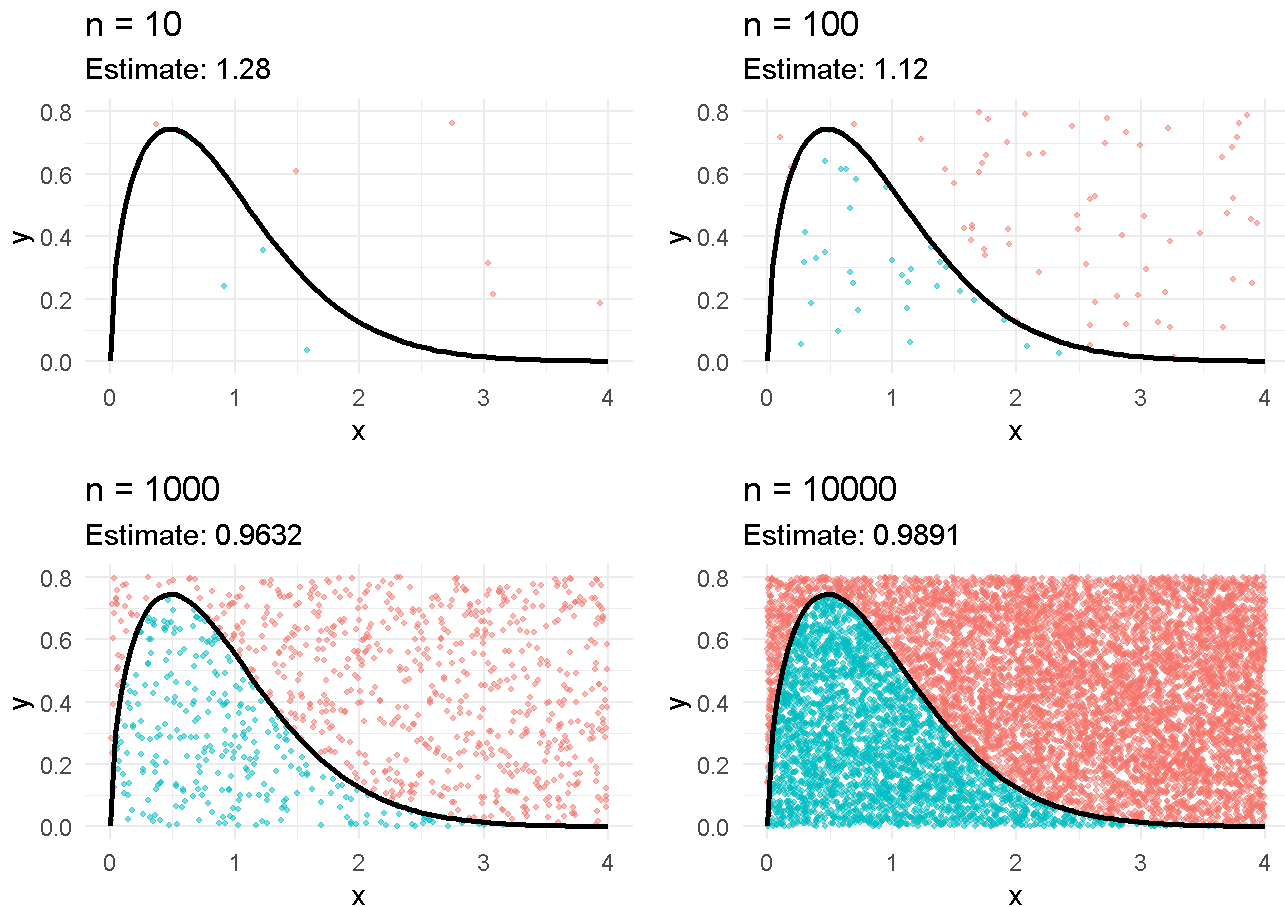
Passenger Traffic Trends at Six Major Global Airports
Annual passenger counts from 2020 to 2025



Monte Carlo Numerical Integration

The exact answer to the numerical integration of this assigned Weibull Distribution is effectively 1.0. Because the function being integrated is a Probability Density Function (PDF), the total area under the curve across its entire range must equal 1.

The small multiple visualization below illustrates the Law of Large Numbers in an intuitive way. At lower resolutions ($n=10$ and $n=100$), the stochastic “noise” of the simulation is highly visible. A single random coordinate has a disproportionate impact on the final result, yielding inaccurate estimates. However, as the resolution increases to $n=10,000$, this jitter evaporates. The points form a dense, high-fidelity fill that perfectly defines the area under the curve, pulling the numerical estimate tightly toward the true value.



Planning and Prompting GenAI Tools

For this section, I developed a plan to wrangle and visualize the newly provided `calcium.csv` dataset. Examining the raw data revealed it is in a “wide” format, tracking measurements across four time points (`y0` to `y3`) for two distinct groups (the second group appended with a `.1`). My plan required generating a subject ID, utilizing `pivot_longer()` to tidy the data into a single measured column, dropping `NA` values resulting from uneven group sizes, and then feeding that clean data into `ggplot2` for a comparative line graph tracking individuals over time.

Comparing the Prompts: When the generic prompt was submitted to ChatGPT (“Create a data visualization for these data. Calcium.csv”), the GenAI tool completely failed to understand the experimental design. It simply calculated the mean of each column and plotted those averages as independent points, completely ignoring the fact that the columns represented repeated time measurements across two distinct experimental groups.

Conversely, when using my plan-based prompt (which explicitly gave the AI the data shape, instructed it to use `pivot_longer`, extract the time/group variables, and map them to a line plot grouped by subject), the GenAI output was an excellent piece of working code. The AI was highly capable of generating the exact syntax for the complex data transformation, but it completely relied on *me* to supply the data-

specific architectural logic to make those choices in the first place. This proves that GenAI is a powerful coding assistant, but a poor independent data analyst when left to its own devices.

This course has been incredibly useful in helping me bridge the gap between abstract mathematical concepts and practical programming. Working heavily in R and mastering the tidyverse has provided an entirely new perspective on data wrangling. Seeing how fluidly data can be reshaped, mutated, and pipelined in R perfectly complements the relational logic I use when building and normalizing SQL databases.

Furthermore, utilizing ggplot2 to manually simulate and visualize probability density functions made the statistics tangibly click in a way that simply reading formulas never did. Finally, putting this all together in Quarto has underscored the importance of reproducible research, ensuring that my code, visualizations, and narrative analysis can exist harmoniously in one professional document.

Appendix A: GenAI Usage

- **Tool Used:** ChatGPT
- **Date of Usage:** 11th April, 2026

Generic Prompt Used: "Create a data visualization for these data. Calcium.csv"

Generated: "Here is the code to create a scatterplot of carat versus price using the diamonds dataset:

I created a visualization showing the average value of each column in your dataset.

What this shows:

- Each point represents the mean (average) of one column (y0, y1, y2, y3, etc.).
- You can quickly compare which variables tend to have higher or lower values overall.
- For example, some columns cluster around ~70–73, while others drop closer to ~65.

If you want something more insightful, I can also create: Boxplots (to show spread and outliers), Time series plots (if rows represent time/order), Correlation heatmaps, Side-by-side comparisons (e.g., y0 vs y0.1). Just tell me 👍

Plan-Based Prompt Used: "Using the diamonds dataset in R, write tidyverse code to first take a stratified random sample of 500 diamonds per clarity level. Then, pipe that into a ggplot scatterplot of carat vs. price colored by clarity. Counteract overplotting by setting alpha to 0.5. Add linear trend lines without standard error bands. Format the y-axis as dollars and apply theme_minimal()."

Plan-Based Response Text Generated: "Here is the code based on your plan:

```
# Load data and add subject ID
calcium_clean <- read.csv("calcium.csv") %>%
```

```
mutate(Subject = row_number()) %>%

# Pivot all 'y' columns to long format
pivot_longer(
  cols = starts_with("y"),
  names_to = "TimeGroup",
  values_to = "Value"
) %>%

# Drop NA rows
drop_na(Value) %>%

# Extract Time (the number right after 'y') and Group (whether it has '.1')
mutate(
  Time = as.numeric(str_extract(TimeGroup, "\\d")),
  Group = ifelse(str_detect(TimeGroup, "\\d\\.1"), "Group 2", "Group 1")
)

# Plot the tidied data
ggplot(calcium_clean, aes(x = Time, y = Value, color = Group, group = Subject)) +
  geom_line(alpha = 0.6) +
  geom_point() +
  labs(
    title = "Calcium Measurements Over Time by Group",
    x = "Time Point",
    y = "Measurement Value",
    color = "Group"
  ) +
  theme_minimal()
```

Appendix B: Code Appendix

```
# This appendix prints all the R code used throughout the document.
```